

UNIDAD 5.

PROGRAMACIÓN MODULAR: FUNCIONES, MÉTODOS Y PROCEDIMIENTOS

**PROGRAMACIÓN
CFGS DAW**

Adaptado por: José Miguel Blázquez
Autores: Carlos Cacho, Raquel Torres, Lionel Tarazon y Fco. Javier Valero

2022/2023

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

 Importante

 Atención

 Interesante

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original. NOTA: Esta es una obra derivada de la obra original realizada por Carlos Cacho y Raquel Torres.

ÍNDICE DE CONTENIDO

1. Introducción: funciones y métodos.....	4
2. Declaración de una función.....	6
3. Llamada a una función.....	9
4. Ámbito de las variables.....	12
5. Parámetros: Paso por valor y por referencia.....	13
6. Devolución de un valor.....	15
7. Diseño e implementación de funciones y métodos.....	16

UD05. PROGRAMACIÓN MODULAR: FUNCIONES Y MÉTODOS

1. INTRODUCCIÓN: FUNCIONES Y MÉTODOS

La mejor forma de crear y mantener un programa grande es construirlo a partir de piezas más pequeñas o módulos. Cada uno de los cuales es más manejable que el programa en su totalidad. Esto, en desarrollo de software, alimenta el “diseño descendente” que se basa en resolver un problema general descomponiéndolo en programas más simples, denominados subproblemas.

⚡ Las **funciones** (subprogramas) son utilizadas para **evitar la repetición de código** en un programa al poder ejecutarlo desde varios puntos de un programa con sólo invocarlo.

El concepto de función es una forma de **encapsular** un conjunto de instrucciones dentro de una declaración específica (llamada generalmente SUBPROGRAMA o FUNCIÓN), permitiendo la descomposición funcional y la diferenciación de tareas, en lo que se denomina **programación modular**, lo cual supone una evolución de la programación estructurada.

Utilidad principal de las funciones:

- Agrupar código que forma una entidad propia o una idea concreta.
- Agrupar código que se necesitará varias veces en un programa, con la misión de no repetir código. Dicho ahorro tiene múltiples ventajas, como facilitar el mantenimiento.
- Dividir el código de un programa grande en subprogramas (funciones), cada uno de ellos especializados en resolver una parte del problema.

Características de las funciones:

- Se definen mediante un nombre único que representa el bloque de código.
- Pueden ser llamadas (ejecutadas) desde cualquier parte del código.
- Se les puede pasar valores para que los procesen de alguna forma.
- Pueden devolver un resultado para ser usado desde donde se les llamó.

¿FUNCIONES, MÉTODOS O PROCEDIMIENTOS?

En Java a las funciones se les llama **métodos**, cuando veamos programación orientada a objetos estudiaremos el porqué. De esta forma, las funciones son realmente un caso particular de método, existiendo otros tipos especiales.

Es muy común entre programadores que se hable indistintamente de estos tres términos sin embargo poseen diferencias fundamentales, veamos una aproximación interesante a estas diferencias:

Funciones:

Las funciones son un conjunto de líneas de código (instrucciones) encapsuladas en un bloque. Suelen recibir parámetros, cuyos valores utilizan para efectuar operaciones, y adicionalmente devuelven un valor. En otras palabras, una función puede recibir parámetros o argumentos (algunas no reciben nada), hace uso de dichos valores recibidos como sea necesario y devuelve un valor usando la instrucción **return**; si no devuelve nada entonces no es una función. En java las funciones usan el modificador **static** al declararlas.

Métodos:

Los métodos y las funciones en Java están en capacidad de realizar las mismas tareas, es decir, son funcionalmente idénticos, pero su diferencia radica en la manera en que hacemos uso de uno u otro (el contexto). Un método también puede recibir valores, efectuar operaciones con estos y devolver un resultado, sin embargo, el método está asociado a un objeto, SIEMPRE. Básicamente podemos decir que un método es una función que pertenece a un objeto o clase, mientras que una función existe por sí sola, sin necesidad de un objeto para ser usada. **Nota:** recordad que, en Java, al ser todo un objeto, las funciones como tal no existen, se les sigue llamando métodos. No es técnicamente correcto emplear el vocablo “funciones” en Java, aunque de manera informal es válido si nos resulta útil para diferenciar el concepto.

Procedimientos:

Los procedimientos son básicamente un conjunto de instrucciones que se ejecutan sin devolver ningún valor, hay quienes dicen que un procedimiento no recibe valores o argumentos, sin embargo en la definición no hay nada que se lo impida. En el contexto de Java un procedimiento es básicamente un método cuyo tipo de retorno es *void*, por lo que no nos obliga a utilizar una sentencia **return**.

```
void procedimiento(int n, String nombre){ //Fíjate en el void
    if(n > 0 && !nombre.equals("")){ //usamos los dos parámetros
        System.out.println("hola " + nombre);
        return; //Si no ponemos este return se mostraría hola y luego adiós
    }
    //También podríamos usar un else en vez del return
    System.out.println("adios");
}
```

Este “procedimiento” no devuelve nada, se limita a realizar unas acciones.

2. DECLARACIÓN DE UNA FUNCIÓN

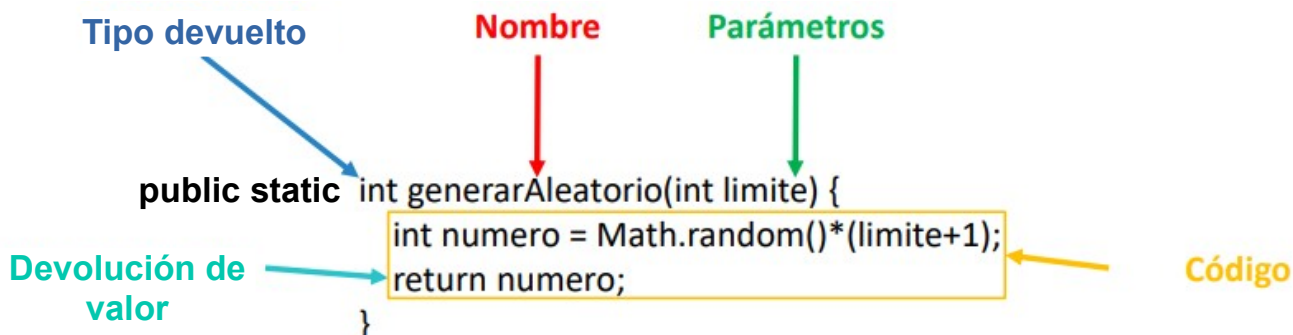
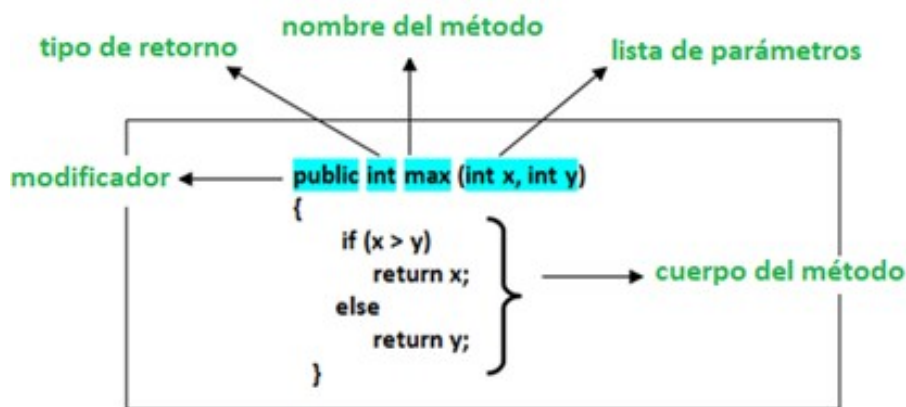
Declarar una función simplemente significa crearla para que luego pueda ser llamada (utilizada) desde otro lugar del código de nuestro programa. Una función se estructura en **cabecera** y **cuerpo**.

La **cabecera** se declara en una sola línea y se compone de:

- **Modificadores de función:** Existen muchos pero los veremos en futuras unidades, especialmente en orientación a objetos. Por ahora solo utilizaremos public static.
- **Tipo devuelto:** El tipo de dato que devolverá la función, como por ejemplo int, double, char, boolean, String, etc. Si la función no devuelve nada se indica mediante void.
- **Nombre de la función:** Identificador único para llamar a la función.
- **Lista de parámetros ó argumentos:** Indica los tipos y nombres de los datos que se le pasarán a la función cuando sea llamada. Pueden ser varios o ninguno. Habitualmente no más de 4 ó 5. Si hay varios se separan por comas.

El **cuerpo** es un bloque de código entre llaves { ... } que se ejecutará cuando desde otra parte del código utilicemos la función.

```
[Modificadores-acceso] tipo_devuelto nombre_función (lista_de_parámetros){  
    // Cuerpo de la función...  
}
```



Ejemplos de funciones:

```
public static void imprimeHolaMundo() {  
    System.out.println("Hola mundo");  
}
```

Este es un ejemplo muy sencillo de una función llamada 'imprimeHolaMundo', que no tiene parámetros de entrada (no hay nada entre los paréntesis) y no devuelve ningún valor (indicado por void). Cuando la llamemos lo único que hará será escribir por pantalla el mensaje "Hola mundo".

```
-----  
public static void imprimeHolaNombre(String nombre) {  
    System.out.println("Hola " + nombre);  
}
```

Esta función se llama 'imprimeHolaNombre', tiene como parámetro de entrada un dato String llamado 'nombre' y no devuelve nada. Cuando la llamemos imprimirá por pantalla el texto "Hola " seguido del String nombre que se le pase como parámetro.

```
-----  
public static int doble(int a) {  
    int resultado = a * 2;  
    return resultado;  
}
```

Esta función se llama 'doble', tiene como parámetro de entrada un dato int llamado 'a' y devuelve un dato de tipo int. Cuando la llamemos calculará el doble de 'a' y lo devolverá (con el return).

```
-----  
public static int multiplica(int a, int b) {  
    int resultado = a * b;  
    return resultado;  
}
```

Esta función se llama 'multiplica', tiene dos parámetros de entrada de tipo int llamados 'a' y 'b' y devuelve un dato de tipo int. Cuando la llamemos calculará a*b y lo devolverá (con el return).

```
-----  
public static double maximo(double valor1, double valor2) {  
    double max;  
    if (valor1 > valor2)  
        max = valor1;  
    else  
        max = valor2;  
    return max;  
}
```

Esta función se llama 'maximo', tiene dos parámetros de entrada de tipo double llamados 'valor1' y 'valor2' y devuelve un dato de tipo double. Cuando la llamemos calculará el máximo entre 'valor1' y 'valor2' y lo devolverá.

```
public static int sumaVector(int v[]) {  
    int suma = 0;  
    for (int i = 0; i < v.length; i++)  
        suma += v[i];  
    return suma;  
}
```

Esta función se llama 'sumaVector', tiene un parámetro de entrada tipo int[] (un vector de int) llamado 'v' y devuelve un dato tipo int. Cuando la llamemos recorrerá el vector 'v', calculará la suma de todos sus elementos y la devolverá.

Es importante saber que **las funciones se declaran dentro de 'class' pero fuera del 'main'**.

```
1  package unidad7;  
2  
3  public class programadeprueba {  
4  
5      public static void imprimeHolaMundo() {  
6          System.out.println("Hola mundo");  
7      }  
8  
9      public static int doble(int a) {  
10         int resultado = a * 2;  
11         return resultado;  
12     }  
13  
14     public static int multiplica(int a, int b) {  
15         int resultado = a * b;  
16         return resultado;  
17     }  
18  
19     public static void main(String[] args) {  
20  
21         // Un programa siempre empieza ejecutándose por la función main.  
22         // Aquí va el código principal de nuestro programa.  
23     }  
24  
25 }  
26  
27
```

En este programa tenemos 4 funciones: **imprimeHolaMundo**, **doble**, **multiplica** y **main**. Sí, el bloque del 'main' donde siempre has programando hasta ahora es en efecto una función, pero un poco especial: **'main' es la función principal, el punto de inicio de un programa**. Es obligatorio que todo programa Java tenga una función **main**. Si te fijas, es una función que recibe como parámetro un String[] (vector de String) y no devuelve nada (aunque podría devolver un int). El por qué de esto lo veremos más adelante. **Las 3 funciones que hemos declarado arriba del main por sí solas no hacen nada, simplemente están ahí esperando a que sean llamadas (utilizadas)**, normalmente desde el propio main.

Veámoslo.

3. LLAMADA A UNA FUNCIÓN

Las funciones pueden ser **invocadas o llamadas** desde cualquier otra función, incluida ella misma. Sí, una función puede llamar a cualquier otra función, y una función puede llamarse a sí misma.

De todos modos **por ahora llamaremos a funciones solo desde la función principal 'main'**. Así es más sencillo de aprender al principio.

Cuando se invoca una función, el **flujo de ejecución** salta a la función (pasándole los parámetros si los hubiera), se ejecutan las instrucciones de la función y por último vuelve al punto que llamó a la función para seguir ejecutándose desde allí.

Las funciones se invocan con su nombre, pasando la lista de parámetros entre paréntesis. Si no tiene parámetros han de ponerse los paréntesis igualmente pero vacíos. Si la función devuelve un valor, para recogerlo hay que asignarlo a una variable o utilizarlo de algún modo (pueden combinarse funciones en expresiones e incluso pasarlo a otras funciones).

Ejemplo utilizando las funciones del apartado anterior:

```
public static void main(String[] args) {
    //No tiene parámetros ni devuelve valor. Imprime "Hola Mundo"
    imprimeHolaMundo();

    //Es habitual llamar a una función
    //y guardar el valor devuelto en una variable
    int a = doble(10); // a = 20    (10*2)
    int b = multiplica(3, 5); // b = 15    (3*5)

    // Pueden pasarse variables como parámetros
    int c = doble(a); // c = 40    (20*2)
    int d = multiplica(a, b); // d = 300    (20*15)

    // Pueden combinarse funciones y expresiones
    int e = doble(4) + multiplica(2,10); // e = 8 + 20

    System.out.println("El doble de 35 es " + doble(35) );
    // "El doble de 35 es 70"

    System.out.println("12 por 12 es " + multiplica(12,12) );
    // "12 por 12 es 144"
}
```

Ejemplo: Programa con una función que suma dos números.

```
4 public class Suma {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int num1, num2, suma;
9
10        System.out.print("Introduce un número: ");
11        num1 = sc.nextInt();
12
13        System.out.print("Introduce otro número: ");
14        num2 = sc.nextInt();
15
16        suma = suma(num1, num2);
17
18        System.out.println("La suma es: " + suma);
19    }
20
21    public static int suma(int n1, int n2) {
22        int suma;
23
24        suma = n1 + n2;
25
26        return suma;
27    }
28
29 }
30
31 }
```

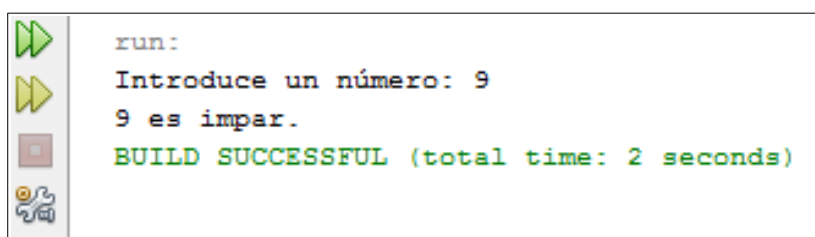
Salida:

```
run:
Introduce un número: 3
Introduce otro número: 4
La suma es: 7
BUILD SUCCESSFUL (total time: 3 seconds)
```

Ejemplo: Programa con una función que determina si un número es par o impar.

```
14 public class ParImpar {
15
16     public static void main(String[] args) {
17         Scanner in = new Scanner(System.in);
18         int num;
19
20         System.out.print("Introduce un número: ");
21         num = in.nextInt();
22
23         if(par(num) == true) // Llamada a la función desde la expresión
24             System.out.println(num + " es par.");
25         else
26             System.out.println(num + " es impar.");
27     }
28
29     public static boolean par(int numero)
30     {
31         boolean par = false;
32
33         if(numero % 2 == 0) // Si el resto es 0 par será 'true' sino 'false'
34             par = true;
35
36         return par;
37     }
38 }
```

Salida:



```
run:
Introduce un número: 9
9 es impar.
BUILD SUCCESSFUL (total time: 2 seconds)
```

4. ÁMBITO DE LAS VARIABLES

Una función solo puede utilizar las variables de ámbito local, es decir, sus propias variables (los parámetros de la cabecera y las variables creadas dentro de la función). Cuando una función se ejecuta se crean sus variables, se utilizan y cuando la función termina se destruyen las variables.

Por todo ello una función no puede utilizar variables que estén fuera de ella, y fuera de una función no es posible utilizar variables de la propia función. A esta característica se le llama **encapsulamiento** y permite que las funciones sean independientes entre sí, facilitando el diseño de programas grandes y complejos, buscando una forma de controlar el *acceso a los datos*.

⚡ Técnicamente sí es posible que una función utilice variables que están fuera de ella, pero eso lo veremos cuando aprendamos Programación Orientada a Objetos.

Veamos un ejemplo:

```
public class Ambitos {  
    public static void funcionPruebas(int num){  
        int y = 7;  
        System.out.println(num * 2);  
  
        System.out.println(x * 2);  
  
        System.out.println(y);  
    }  
  
    public static void main(String[] args) {  
        int x = 5;  
  
        for(int i = 0; i < 10; i++){  
            System.out.println(i);  
        }  
  
        System.out.println(i);  
        System.out.println(x);  
    }  
}
```

Ámbito local de "funcionPruebas"

Error... x no se ve en este ámbito

Ámbito local de "main"

Ámbito de bloque

Error... i no se ve en este ámbito

5. PARÁMETROS: PASO POR VALOR Y POR REFERENCIA

Existen dos tipos de parámetros y es importante comprender la diferencia.

- **Parámetros de tipo simple (paso por valor):** Como int, double, boolean, char, etc. En este caso **se pasan por valor**. Es decir, **el valor se copia al parámetro** y por lo tanto si se modifica dentro de la función esto no afectará al valor fuera de ella porque **son variables distintas**.

```
public static void main(String[] args) {  
    int a = 10;  
    System.out.println("Valor inicial de a: " + a);  
    // a vale 10  
    imprime_doble(a); // Se le pasa el 10 a la función  
    System.out.println("Valor final de a: " + a);  
    // a sigue valiendo 10  
}  
  
// El parámetro 'a' es independiente de la 'a' del main.  
// ¡Son variables distintas!  
  
public static void imprime_doble(int a) {  
    // Se copia el valor 10 a esta nueva 'a'  
    a = 2 * a;  
    // Duplica el valor de 'a' dentro de la función, no fuera  
    System.out.println("Valor de a en la función: " + a);  
    // 'a' vale 20  
}
```

SALIDA:

```
run:  
Valor inicial de a: 10  
Valor de a en la función: 20  
Valor final de a: 10
```

- **Parámetros de tipo objeto (paso por copia de referencias):** con objetos de tipo String, los Arrays, etc. En este caso no se copia el contenido del objeto sino que **se le pasa a la función una COPIA de la referencia al objeto original (un puntero)**. Por ello **desde la función se accede directamente al objeto** que se encuentra fuera. Los cambios que hagamos dentro de la función afectarán al objeto.

```
// Suma x a todos los elementos del vector v
public static void suma_x_al_vector(int v[], int x) {
    for (int i = 0; i < v.length; i++)
        v[i] = v[i] + x;
}

public static void main(String[] args) {
    int v[] = {0, 1, 2, 3};
    System.out.println("Vector antes: " + Arrays.toString(v));
    suma_x_al_vector(v, 10);
    System.out.println("Vector después: " + Arrays.toString(v));
}
```

SALIDA:

```
run:
Vector antes: [0, 1, 2, 3]
Vector después: [10, 11, 12, 13]
```

IMPORTANTE: Como un parámetro de tipo objeto es una referencia al objeto String o Array que está fuera de ella, si se le asigna otro objeto se pierde la referencia y ya no se puede acceder al objeto fuera de la función. Aunque Java permite hacerlo, **no se aconseja hacerlo**.

```
// Asignamos a x un nuevo vector, por lo que x dejará de apuntar al
vector original.
```

```
// ¡El vector original no cambia! Simplemente ya no podemos acceder
a él desde x porque se ha perdido la referencia a dicho objeto.
```

```
public static void funcion1(int x[]) {
    x = new int[10];
    // x apuntará a un nuevo vector, el original queda intacto
    // lo que hagamos aquí con x no afectará al vector original
}
```

```
// Lo mismo sucede en este ejemplo
// perdemos la referencia al String original
```

```
public static void funcion2(String x) {
    x = "Hola";
    // x apuntará a un nuevo String, el original queda intacto
    // lo que hagamos aquí con x no afectará al String original
}
```

NO SE ACONSEJA HACER ESTE TIPO DE COSAS.

6. DEVOLUCIÓN DE UN VALOR

⚡ Los métodos pueden devolver valores de tipo básico o primitivo (int, double, boolean, etc.) y también de tipo objeto (Strings, arrays, etc.).

En todos los casos es el comando **return** el que realiza esta labor. En el caso de arrays y objetos, devuelve una referencia a ese array u objeto.

Sintaxis: `return expresion;`

Ejemplos:

```
public static double cubo (double x) {  
    return x*x*x;  
}
```

```
public static double cubo (double x) {  
    int cubo = x*x*x;  
    return cubo;  
}
```

¡Hacen lo mismo! Fíjate en cómo se emplea la sentencia **return**.

En algunas ocasiones no es necesario que el método estático tenga que devolver un valor al finalizar su ejecución. En este caso, el tipo de dato que debe indicar en la cabecera de declaración del método es el tipo **void** y la sentencia **return** debe omitirse.

Si no hay sentencia **return** dentro de un método, su ejecución continúa hasta que se alcanza el final del método y entonces se devuelve la secuencia de ejecución al lugar dónde se invocó al método.

Un método cuyo tipo de retorno no es **void** necesita siempre devolver algo. Si el código de un método contiene varias sentencias if, hay que asegurarse de que cada una de las posibles opciones devuelve un valor. En caso contrario, se generaría un error de compilación. Por ejemplo:

```
public static boolean esPositivo(int x) {  
    if (x<0) return false;  
    if (x>0) return true;  
    // Error: retorno perdido si x es igual a cero.  
}  
}
```

Ejemplo de intento de compilación del código anterior:

```
$>javac PruebaPositivo.java
pruebaPositivo.java:14: missing return statement
    }
    ^
```

7. DISEÑO E IMPLEMENTACIÓN DE FUNCIONES Y MÉTODOS

A la hora de crear un método o una función, es importante que sigamos unos pasos con el objetivo de garantizar un buen trabajo en su construcción:

1. Describir lo que el método debe hacer.
2. Determinar los parámetros de entrada del método, es decir, qué datos debemos proporcionarle al método para que pueda realizar su trabajo.
3. Determinar los tipos de datos de las entradas (son números enteros, decimales, Strings...)
4. Determinar lo que debe devolver el método y el tipo del valor devuelto, es decir, qué información debe proporcionar cuando termine su trabajo.
5. Escribir las instrucciones que forman el cuerpo del método.
6. Prueba del método: diseñar distintos casos de prueba y comprobar si funciona correctamente.